

Chapter 1

Methodology

1.1 Introduction

Chapter 3 stated eight functional requirements that partition the query level control flow of CAEM, from pre-routing confidence estimation through ablation instrumentation. This chapter operationalises those requirements into a working system. Each mechanism described here corresponds to a concrete block of the reference implementation, and each equation is the equation the code evaluates at run time. The chapter is constructed so that two kinds of reader are served: one who reads linearly and assembles a complete mental model of CAEM, and one who reads only the figures and captions and still recovers the architecture.

The CAEM pipeline executes in eight stages for every arriving query, bracketed by an outer self-improvement loop that re-verifies the episodic memory and fine-tunes the base model at fixed cycle boundaries. Stage 1 estimates, in a single encoder forward pass, how confidently the base model can answer the query using its internal parameters alone. Stage 2 retrieves the single nearest stored episode from the episodic memory. Stage 3 dispatches the query to one of three processing tiers on the basis of a blended score over pre-route confidence and the retrieved episode’s stored composite, with a safety override that forces external grounding whenever pre-route confidence falls below a minimum floor. Stage 4 is the chosen tier. Tier 1 returns a stored answer untouched, Tier 2 generates zero shot from the base model, and Tier 3 generates under grounding against a preprocessed Wikipedia passage index. Stage 5 subjects every Tier 2 and Tier 3 answer to a nine signal verifier whose composite score is the memory’s storage criterion. Stage 6 emits one of four categorical decisions: STORE, DEFERRED, ABSTAIN, or DISCARD. Stage 7 commits the episode to memory, flags it for training eligibility, or returns a principled refusal, as dictated by Stage 6. Stage 8 runs at cycle boundaries rather than per query: it re-verifies every stored entry under the freshly fine-tuned weights, prunes entries whose composite has fallen below a retroactive threshold, curates a training pool from the survivors, fine-tunes the base model under an L2 anchor regulariser, and rolls the cycle back if a held out retention probe falls below the forgetting bound.

The eight stages correspond one to one with Functional Requirements FR1 through FR7 of Chapter 3, with FR8 (ablation and evaluation instrumentation) realised by the experimental harness described in Section ?? rather than by a pipeline stage. Section 1.2 presents the complete architecture in a single figure, introduces the notation used throughout the rest of the chapter, and walks a single fabricated query end to end through all eight stages to ground the abstract description in a concrete trace. Sections 1.3 through 1.9

then treat the stages one by one at implementation fidelity. Sections ?? and ?? give the reproducibility-critical implementation details and the experimental protocol under which Chapter 5 reports results. Section ?? closes the chapter.

1.2 System Overview

Figure 1.1 shows the per-query path through CAEM on a single page. The between-cycle self-improvement loop that re-verifies memory and fine-tunes the base model has its own diagram in Section 1.9 (Figure ??). Every symbol used in the rest of the chapter is defined in Table 1.1, and a single fabricated query is traced end to end through all eight stages in Example so that the reader acquires the mechanisms in concrete form before meeting them in the abstract.

1.2.1 Stage-by-stage narrative

Stage 1: Pre-route confidence. The base model encoder is run once over the query, yielding two scalars: a token-probability aggregate u_{token} over a short greedy prefix of the candidate answer, and a convergence ratio C_{conv} measuring how quickly stacked activations stabilise through the encoder layers. The two signals are fused into the pre-route confidence u_{pre} . A single scalar temperature is fit offline at the end of Cycle 0 by L-BFGS minimisation of the negative log-likelihood on a held-out calibration split, written to the pipeline configuration, and held fixed for all subsequent cycles; the calibration is therefore a one-shot external step rather than an inline operation inside the per-query pipeline. u_{pre} is the only quantity Stage 3 uses to decide whether the query is eligible for the cheap retrieval tier.

Stage 2: Episodic memory retrieval. The query embedding, produced by the sentence encoder, is searched against the FAISS-backed episodic memory for its single nearest neighbour e^* , returning its cosine similarity and its previously stored composite $u_{\text{stored}}(e^*)$. No generation has yet occurred.

Stage 3: Three-tier router. A combined score $S(q) = \alpha \text{sim}(q, e^*) + (1 - \alpha) u_{\text{stored}}(e^*)$ is computed. A safety override evaluated before the score checks whether u_{pre} has fallen below a minimum floor of 0.60; if so, the query is forced to Tier 3 regardless of memory content. Otherwise, Tier 1 is taken if $S(q) \geq 0.90$, Tier 2 if $\text{sim}(q, e^*) \geq 0.75$ and Tier 1 was not taken, and Tier 3 otherwise.

Stage 4: Tier execution. Tier 1 returns the stored answer attached to e^* directly and emits a STORE outcome without passing through the verifier, because $u_{\text{stored}}(e^*)$ already cleared the composite threshold at the original storage event and entries are re-verified at cycle boundaries. Tier 2 generates an answer zero shot from the base model given the query alone. Tier 3 retrieves the top-5 passages from the DPR-preprocessed Wikipedia index, cross-encoder reranks them to top-3, and conditions Flan-T5-Large on those passages before decoding the answer.

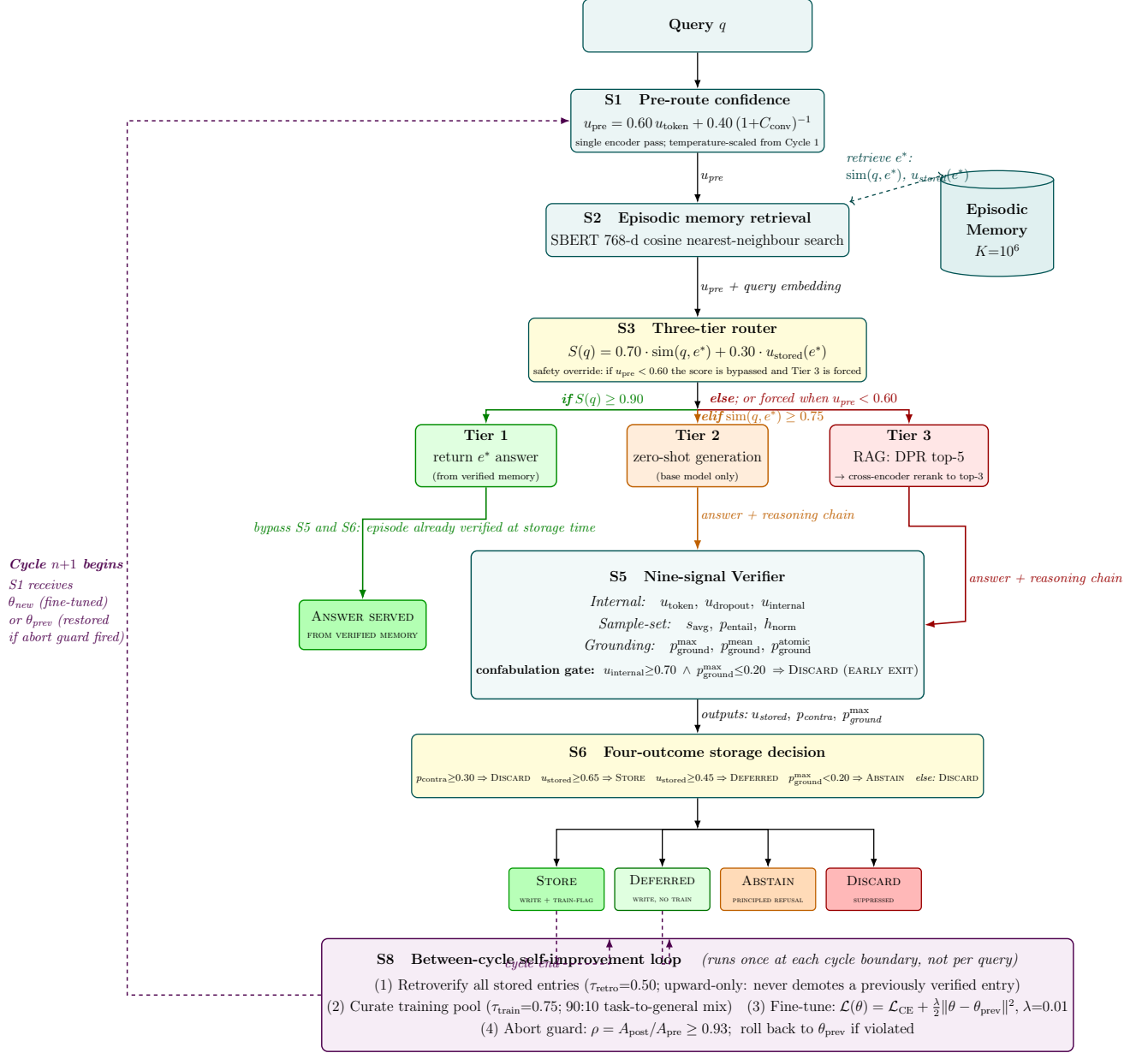


Figure 1.1: CAEM pipeline overview (S1–S8): each query passes through confidence estimation, memory retrieval, routing, tier execution, verification, and storage decision. Stage 8 runs once per cycle boundary for retroverification and L_2 -anchored self-improvement.

Table 1.1: Notation used throughout Chapter 4. All probability-style signals lie in $[0, 1]$ and are either native probabilities (NLI outputs) or normalised into $[0, 1]$.

Symbol	Range	Description
<i>Confidence signals</i>		
u_{token}	$[0, 1]$	Token-probability aggregate over greedy prefix.
C_{conv}	$\mathbb{R}_{\geq 0}$	Encoder layer convergence ratio.
u_{pre}	$[0, 1]$	Pre-route confidence (Stage 1 output, temperature-scaled after Cycle 0).
u_{dropout}	$[0, 1]$	Monte Carlo dropout disagreement over K_D stochastic passes.
u_{internal}	$[0, 1]$	Internal confidence fusion of u_{token} and u_{dropout} .
s_{avg}	$[0, 1]$	Mean pairwise cosine similarity over M sampled reasoning chains.
p_{entail}	$[0, 1]$	Mean NLI entailment probability chain-to-answer.
h_{norm}	$[0, 1]$	Normalised semantic entropy over K_S chains clustered by bidirectional entailment.
$p_{\text{ground}}^{\text{max}}$	$[0, 1]$	Maximum entailment answer-to-passage over top-3 reranked passages.
$p_{\text{ground}}^{\text{mean}}$	$[0, 1]$	Mean entailment answer-to-passage over top-3 reranked passages.
$p_{\text{ground}}^{\text{atomic}}$	$[0, 1]$	Weakest-link entailment over decomposed atomic facts.
p_{contra}	$[0, 1]$	Maximum contradiction probability over top-3 reranked passages.
u_{stored}	$[0, 1]$	Composite verifier confidence written to memory.
<i>Routing</i>		
$\text{sim}(q, e)$	$[-1, 1]$	Cosine similarity in sentence-embedding space.
e^*	—	Single nearest stored episode to q .
$S(q)$	$[0, 1]$	Combined routing score $\alpha \text{sim}(q, e^*) + (1 - \alpha) u_{\text{stored}}(e^*)$.
α	0.70	Routing similarity blend weight.
<i>Thresholds</i>		
τ_{store}	0.65	Composite threshold for STORE.
τ_{defer}	0.45	Composite threshold for DEFERRED.
τ_{abs}	0.20	Grounding-floor threshold for ABSTAIN.
τ_{veto}	0.30	Contradiction-veto threshold.
τ_{retro}	0.50	Retroactive re-verification prune threshold.
τ_{train}	0.75	Training-eligibility threshold.
τ_{forget}	0.93	Forgetting-bound (cycle retention).
<i>Self-improvement and memory</i>		
$\theta, \theta_{\text{prev}}$	—	Model parameters at current step and at cycle start (anchor).
λ	0.01	L2-anchor coefficient.
ρ	$\mathbb{R}_{\geq 0}$	Retention ratio $\text{MMLU}_{\text{post}}/\text{MMLU}_{\text{pre}}$.
K_{mem}	10^6	Episodic memory capacity.
N_{train}	20,000	FAISS exact-to-IVF-PQ promotion point.
n_{list}	4096	IVF coarse centroid count (memory index).
K_D	5	MC dropout passes for u_{dropout} .
K_S	10	Semantic entropy sample count for h_{norm} .
M	3	Chain count for s_{avg} .

Stage 5: Unified nine-signal verification. Tier 2 and Tier 3 answers are passed through a single verifier that computes nine signals in three families. The internal family $\{u_{\text{token}}, u_{\text{dropout}}, u_{\text{internal}}\}$ measures the model’s self-assessed confidence. The sample-set family $\{s_{\text{avg}}, p_{\text{entail}}, h_{\text{norm}}\}$ measures cross-chain agreement. The grounding family $\{p_{\text{ground}}^{\text{max}}, p_{\text{ground}}^{\text{mean}}, p_{\text{ground}}^{\text{atomic}}\}$ measures how well the answer is entailed by external Wikipedia evidence. The atomic variant is computed by prompting the base model to decompose the answer into a short list of standalone factual claims, parsing the numbered list, running an NLI pass between each atom and each of the top-three reranked passages, and recording the weakest-link entailment across atoms; this catches partially-correct answers in which one atom is supported while another is silently fabricated, a failure mode the max and mean aggregates cannot distinguish. The verifier also records p_{contra} , the maximum contradiction probability across the same passages. A confabulation gate evaluated before the composite discards the answer immediately whenever internal confidence is high and grounding is effectively absent. Surviving answers are combined into the composite u_{stored} by a six-weight convex combination.

Stage 6: Four-outcome storage decision. Given u_{stored} , p_{contra} , and $p_{\text{ground}}^{\text{max}}$, the system emits one of four categorical decisions. A contradiction veto at $p_{\text{contra}} \geq \tau_{\text{veto}}$ forces DISCARD. Otherwise the composite is thresholded: $u_{\text{stored}} \geq \tau_{\text{store}}$ emits STORE; $u_{\text{stored}} \geq \tau_{\text{defer}}$ emits DEFERRED; $p_{\text{ground}}^{\text{max}} < \tau_{\text{abs}}$ emits ABSTAIN; all other cases emit DISCARD.

Stage 7: Memory write and output. A STORE decision writes the full episode to memory (question, chain, answer, embedding, nine-signal record, composite, contradiction probability, cycle index, decision) and flags it as training-eligible provided $u_{\text{stored}} \geq \tau_{\text{train}}$. A DEFERRED decision writes the episode but withholds the training flag, pending promotion or pruning at the next cycle boundary. ABSTAIN returns a principled refusal and writes nothing. DISCARD suppresses the generation entirely.

Stage 8: Between-cycle self-improvement. At the end of every cycle, the full verifier is re-run over every stored episode under the updated weights. Updates are applied asymmetrically: re-verification can only *raise* a stored composite, in which case the full nine-signal record is replaced; it never writes back a lower composite than the entry’s original score, so that a single bad cycle cannot silently demote a previously well-verified episode. Irrecoverable entries are instead handled by pruning at the retroactive threshold τ_{retro} . A training pool is then curated from training-eligible stores, mixed with general-domain examples in a 90:10 ratio, and the base model is fine-tuned under an L_2 -anchor regulariser with coefficient λ . A single per-cycle MMLU evaluation computes the retention ratio $\rho = \text{MMLU}_{\text{post}}/\text{MMLU}_{\text{pre}}$; if $\rho < \tau_{\text{forget}}$, the post-cycle weights are discarded and θ_{prev} is restored, otherwise they are committed. The same pass feeds the RET axis of the Composite Evaluation Score, so one broad-capability probe simultaneously serves as the rollback trigger and as the reported retention metric. The loop repeats for ten cycles, with Cycle 0 serving as the untrained baseline and as the one-shot calibration fit for the Stage 1 temperature scaler.

1.2.2 Worked example

[enhanced, breakable, width=colback=teal!3, colframe=teal!55!black, colbacktitle=teal!55!black, coltitle=white, fonttitle=, title=Example End-to-end trace through CAEM on two contrasting queries, boxrule=0.6pt, arc=3pt, left=14pt, right=14pt, top=10pt, bottom=12pt, titlerule=0pt, attach boxed title to top left=yshift=-2mm, xshift=10pt, boxed title style=arc=2pt, boxrule=0pt, colback=teal!55!black, toptitle=2pt, bottomtitle=2pt, enlarge top at break by=0mm, enlarge bottom at break by=0mm,]

■ Query A (Tier 1 cheap path)

Consider $q = \text{“Who discovered radium?”}$ arriving at Cycle 3, with the memory already populated by roughly 17,000 entries from earlier cycles.

Stage 1 | Pre-route confidence. One encoder pass produces $u_{\text{token}} = 0.82$ over the greedy prefix and $C_{\text{conv}} = 0.41$, giving

$$u_{\text{pre}} = 0.60 \cdot 0.82 + 0.40 \cdot \frac{1}{1.41} = 0.776$$

after temperature scaling fit at the end of Cycle 0. The safety override is not triggered.

Stage 2 | Memory retrieval. SBERT nearest neighbour returns e^* with $\text{sim}(q, e^*) = 0.93$ and $u_{\text{stored}}(e^*) = 0.86$.

Stage 3 | Three-tier router. The combined score is

$$S(q) = 0.70 \cdot 0.93 + 0.30 \cdot 0.86 = 0.909 \geq 0.90,$$

so the router dispatches to Tier 1.

Stage 4 | Tier 1 execution. The stored answer attached to e^* , “*Marie Curie and Pierre Curie*”, is returned directly without invoking the verifier or any generation. This is the cheap path the verified-memory investment is designed to enable.

Stages 5–7 | Verifier bypassed. Tier 1 entries carry a composite from their original storage event and are re-verified between cycles; the outcome is Store-equivalent and the user is served immediately from verified memory.

■ Query B (Tier 2 full verify-and-decide path, at the end of Cycle 3)

A later query $q' = \text{“Who first synthesised aspirin?”}$ routes instead to Tier 2; the zero-shot generator produces “*Felix Hoffmann*”. The verifier’s nine-signal record is:

<i>Internal</i>		<i>Sample-set</i>		<i>Grounding</i>			
u_{internal}	0.78	s_{avg}	0.74	$p_{\text{ground}}^{\text{max}}$	0.88	<i>Veto</i>	
		p_{entail}	0.81	$p_{\text{ground}}^{\text{mean}}$	0.71	p_{contra}	0.04
		h_{norm}	0.22	$p_{\text{ground}}^{\text{atomic}}$	0.65		

Confabulation gate. Not triggered, since $p_{\text{ground}}^{\text{max}} = 0.88 \gg 0.20$.

Composite.

$$\begin{aligned} u_{\text{stored}} &= 0.30 (0.71) + 0.15 (0.65) + 0.15 (0.81) \\ &\quad + 0.15 (0.74) + 0.15 (0.78) + 0.10 (1 - 0.22) \\ &= \mathbf{0.7335}. \end{aligned}$$

Decision. Because $u_{\text{stored}} = 0.7335 \geq \tau_{\text{store}} = 0.65$ and $p_{\text{contra}} = 0.04 < \tau_{\text{veto}} = 0.30$, the decision is **Store**. Because $u_{\text{stored}} < \tau_{\text{train}} = 0.75$, it is *not* flagged as training-eligible; it will be reconsidered at the next cycle’s retroactive re-verification pass.

The rest of Chapter 4 fills in each stage at the fidelity required to re-implement it from this text. Sections 1.3 through 1.9 follow the eight-stage order of Figure 1.1.

1.3 Pre-Routing Confidence Estimation

Stage 1 produces a calibrated scalar $u_{\text{pre}} \in [0, 1]$ that estimates how confidently the base model could answer the incoming query from its internal parameters alone. The stage runs before any generation, any retrieval, and any storage decision, so that its cost (a single encoder forward pass) is paid uniformly across all three downstream tiers, including Tier 1 queries that will ultimately be served without generation. Two design commitments shape the computation. First, the estimate must be obtainable without running the decoder, because the whole point of Stage 1 is to decide whether the decoder needs to run at all; this rules out any signal that requires sampling multiple completions. Second, the estimate must discriminate reliably at its low end, since u_{pre} is consumed both by the combined routing score of Stage 3 (Section 1.5) and by the safety override that forces Tier 3 retrieval when $u_{\text{pre}} < 0.60$. Confidence signals that are well-calibrated in the high region but noisy near zero are unsuitable for this role, because a safety override is a veto on cheap paths and its false positives cost nothing while its false negatives cost hallucinations.

Stage 1 achieves both properties by combining two complementary signals drawn from the same encoder forward pass. The first signal, u_{token} , is a token-level aggregate over the greedy-decoded answer prefix; it captures lexical confidence in the surface form the model would have produced. The second signal, C_{conv} , is an encoder-internal convergence ratio measured over the query representation alone; it captures representational stability before any token is emitted. The two signals disagree systematically in the failure mode Stage 1 most needs to catch: a query that is superficially familiar in phrasing but substantively out of distribution yields a high u_{token} over whatever surface form the greedy decoder commits to, but a representation that has not yet settled through the encoder stack yields an elevated C_{conv} . Fusing both signals ensures that u_{pre} is suppressed in exactly this regime.

Token-probability aggregate. The base model generates a greedy prefix of at most 32 tokens and records the per-step conditional log-probability of each emitted token. Generation terminates at the EOS token or at the cap, whichever comes first, and EOS itself is excluded from the aggregate. Writing $\ell_1, \dots, \ell_n$ for the non-EOS log-probabilities of the emitted prefix, the aggregate is their geometric mean expressed as an exponentiated

arithmetic mean of log-probabilities,

$$u_{\text{token}} = \exp\left(\frac{1}{n} \sum_{i=1}^n \ell_i\right) = \left(\prod_{i=1}^n p(y_i \mid y_{<i}, q)\right)^{1/n}, \quad (1.1)$$

so that $u_{\text{token}} \in (0, 1]$ with the natural interpretation of mean per-token probability. The geometric-mean form is chosen over the arithmetic mean of probabilities because log-probabilities decompose additively across steps, making the aggregate insensitive to prefix length: a 10-token and a 30-token answer with the same mean per-token probability receive the same score. A prefix cap of 32 tokens is imposed so that Stage 1 cost remains bounded even on queries whose greedy completion runs long; empirically the cap is reached only on enumerations, and the confidence signal derived from the first 32 tokens is a faithful proxy for the full answer.

Encoder convergence ratio. The encoder’s L intermediate hidden states $h_1, \dots, h_L$ are captured during the same forward pass; the input embedding layer h_0 is excluded so that the ratio measures contextualisation rather than the tokenizer’s inductive bias. The stack is split at its midpoint into an early half $\{h_1, \dots, h_{L/2}\}$ and a late half $\{h_{L/2+1}, \dots, h_L\}$, and the convergence ratio is defined as the ratio of scalar activation variances,

$$C_{\text{conv}} = \frac{\text{Var}(h_1, \dots, h_{L/2})}{\text{Var}(h_{L/2+1}, \dots, h_L) + \varepsilon}, \quad \varepsilon = 10^{-8}, \quad (1.2)$$

where variance is computed over the flattened concatenation of each half’s hidden-state tensors (no pooling across the sequence dimension, so long queries contribute proportionally more samples). A well-converged representation is one in which the late layers vary much less than the early layers as the encoder progressively stabilises the meaning of the query, producing $C_{\text{conv}} > 1$. A poorly-converged representation, the situation in which the encoder is still reshaping the query by its final layers, produces C_{conv} near unity or below. The exact midpoint split is a deliberate choice over alternatives such as first-third/last-third: on Flan-T5-Large we observed that the most diagnostic contrast is between the query-understanding layers (the first half, which performs lexical and syntactic integration) and the query-resolution layers (the second half, which contextualises against factual associations).

Fusion and temperature calibration. The two signals are fused into a raw pre-route confidence by the linear combination specified in Equation ?? of Chapter 3, reproduced here with the transformation that maps C_{conv} into the unit interval made explicit,

$$\tilde{u}_{\text{pre}} = w_{\text{tok}} \cdot u_{\text{token}} + w_{\text{conv}} \cdot \frac{1}{1 + C_{\text{conv}}}, \quad w_{\text{tok}} = 0.60, \quad w_{\text{conv}} = 0.40, \quad (1.3)$$

with $\tilde{u}_{\text{pre}}$ then clipped to $[0, 1]$ to absorb the floating-point tail of either component. The clipping is mechanically necessary because the finite precision of the softmax and the hidden-state variance can each produce values marginally outside the unit interval, but in practice it fires only on degenerate queries where it is already immaterial which tier executes.

The raw estimate $\tilde{u}_{\text{pre}}$ is a biased estimator of empirical correctness: language models are systematically over-confident, and a weighted sum of two such signals is over-confident

in the same direction. To remove the bias, $\tilde{u}_{\text{pre}}$ is passed through a single-parameter temperature scaler fit once, offline, at the end of Cycle 0 on a held-out calibration split of 500 queries never used for training or evaluation. The scaler operates in logit space:

$$u_{\text{pre}} = \sigma\left(\frac{\text{logit}(\tilde{u}_{\text{pre}})}{T}\right), \quad \sigma(x) = \frac{1}{1 + e^{-x}}, \quad (1.4)$$

where T is the scalar temperature. This is the standard post-hoc recalibration of Guo et al. [?], written in sigmoid form for a scalar probability rather than in softmax form for a multi-class logit vector; the two are equivalent and the choice is notational. T is fit by minimising the binary negative log-likelihood of the calibration split under L-BFGS, using empirical exact-match correctness as the binary target:

$$T^* = \arg \min_{T>0} -\frac{1}{N} \sum_{i=1}^N \left[y_i \log \sigma(\text{logit}(\tilde{u}_{\text{pre}}^{(i)})/T) + (1-y_i) \log (1 - \sigma(\text{logit}(\tilde{u}_{\text{pre}}^{(i)})/T)) \right]. \quad (1.5)$$

NLL is used as the training objective in place of the expected calibration error (ECE) reported in Chapter 3’s acceptance criterion, because ECE is a binned statistic and not differentiable; minimising NLL produces a temperature that minimises ECE in practice while remaining tractable for L-BFGS. The fitted T^* is written to the pipeline configuration and held fixed for all subsequent cycles; Chapter 3’s NFR on calibration stability ($\text{ECE} \leq 0.05$ after Cycle 1) is an acceptance check on this fit rather than an inline regularisation.

Algorithm 1.1 summarises the stage. The algorithm assumes that the temperature T has been fit; in the very first cycle the temperature defaults to unity and the sigmoid-of-logit identity leaves the raw estimate unchanged, so the calibration is a strict improvement applied from Cycle 1 onward rather than a correctness dependency.

Algorithm 1.1 Stage 1 pre-route confidence u_{pre} .

Require: Query q ; base model f_θ ; weights $w_{\text{tok}} = 0.60$, $w_{\text{conv}} = 0.40$; temperature T ; prefix cap $M = 32$.

Ensure: Calibrated scalar $u_{\text{pre}} \in [0, 1]$.

- 1: $(y_{1:n}, \ell_{1:n}, h_{0:L}) \leftarrow$ greedy decode of $f_\theta(q)$ up to M tokens, recording emitted token log-probabilities ℓ_i and encoder hidden states $h_{0:L}$
 - 2: exclude EOS from $\ell_{1:n}$; if empty, **return** 0 $\triangleright$ safety-override eligible
 - 3: $u_{\text{token}} \leftarrow \exp(\frac{1}{n} \sum_i \ell_i)$ $\triangleright$ geometric mean of token probabilities
 - 4: $\text{Var}_{\text{early}} \leftarrow \text{Var}(h_{1:L/2})$; $\text{Var}_{\text{late}} \leftarrow \text{Var}(h_{L/2+1:L})$ $\triangleright$ flattened across seq_len $\times$ hidden_dim
 - 5: $C_{\text{conv}} \leftarrow \text{Var}_{\text{early}} / (\text{Var}_{\text{late}} + 10^{-8})$
 - 6: $\tilde{u}_{\text{pre}} \leftarrow \text{clip}(w_{\text{tok}} u_{\text{token}} + w_{\text{conv}} (1 + C_{\text{conv}})^{-1}, 0, 1)$
 - 7: $u_{\text{pre}} \leftarrow \sigma(\text{logit}(\tilde{u}_{\text{pre}})/T)$ $\triangleright$ identity when $T = 1$
 - 8: **return** u_{pre}
-

The scalar u_{pre} produced by Algorithm 1.1 is the sole quantity Stage 3 uses to evaluate the safety override, and one of two quantities (the other being the stored composite of the retrieved episode) that enter the combined routing score. Its production is the only computation in the entire pipeline that is paid on every query regardless of tier, which sets the cost floor of CAEM at one encoder forward pass per query and motivates the single-pass, decoder-free design of this section.

1.4 Episodic Memory

The episodic memory is the persistent store that CAEM consults at Stage 2 of every query, writes to at Stage 7 after a STORE or DEFERRED outcome, and re-verifies wholesale at Stage 8 between cycles. It is the central artefact that accumulates across cycles: a well-designed memory makes the cheap Tier 1 path increasingly attractive, while a poorly curated memory either forces unnecessary retrieval (by failing to serve near-duplicates of previous queries) or pollutes the Tier 1 path with low-quality answers. This section specifies what is stored, how it is indexed, how novelty and capacity are managed, and how retrieval feedback updates stored confidence between cycles.

Data schema. Each memory entry is an EPISODICENTRY instance holding five immutable content fields and a mutable quality-and-usage record, summarised in Table 1.2. The immutable fields (question, reasoning chain, answer, 768-dimensional embedding, storage cycle, and creation timestamp) are set at storage time and never modified afterwards. This invariance is load-bearing: Stage 8 derives the fine-tuning pool from these fields, and mutating them mid-cycle would mean training against a moving target with no reliable audit trail of which (query, chain, answer) triples actually updated the model. The mutable record carries the full nine-signal verifier output, the contradiction probability p_{contra} , the categorical Stage 6 decision, the confabulation-gate flag, the composite u_{stored} , and three usage fields (retrieval count, running success rate, retroverified flag). The composite u_{stored} is defined by the six-weight convex combination of Chapter 3, Equation 3.5, repeated here for reference,

$$u_{\text{stored}} = 0.30 p_{\text{ground}}^{\text{mean}} + 0.15 p_{\text{ground}}^{\text{atomic}} + 0.15 p_{\text{entail}} + 0.15 s_{\text{avg}} + 0.15 u_{\text{internal}} + 0.10 (1 - h_{\text{norm}}), \quad (1.6)$$

with $(1 - h_{\text{norm}})$ appearing because semantic entropy is a disagreement signal and the composite is a confidence. Storage happens only if $u_{\text{stored}} \geq \tau_{\text{defer}}$ at Stage 6, so every entry in memory carries a positive prior that survives a two-threshold screen; this prior is the quantity Stages 3 and 5 subsequently consume.

Embedding model and similarity metric. Embeddings are produced by the Sentence-BERT `all-mpnet-base-v2` model, which maps arbitrary queries into a 768-dimensional vector space. Every embedding is L2-normalised to unit length at the moment of production, so that the inner product between two embeddings equals their cosine similarity,

$$\text{sim}(q, e) = \hat{q}^\top \hat{e}, \quad \hat{v} = v / \|v\|_2, \quad (1.7)$$

which is the property FAISS’s METRIC_INNER_PRODUCT relies on for cosine-ordered nearest-neighbour results. Normalisation is applied once at storage time and once at query time; it is never re-applied after retrieval, so that similarities retrieved from FAISS can be consumed directly by Stage 3’s routing score without further rescaling.

Index structure and size-adaptive promotion. The memory is backed by a FAISS index wrapped in an IndexIDMap so that entries can be removed in place during pruning. For the first phase of operation the backend is a flat inner-product index, which performs exact cosine nearest-neighbour search at $O(N)$ per query. When the number of stored vectors first reaches a promotion threshold $N_{\text{train}} = 20,000$, the index is rebuilt in place as a product-quantised inverted file: an IndexIVFPQ with $n_{\text{list}} = 4096$ coarse centroids,

Table 1.2: Fields of an EPISODICENTRY. Immutable content is the substrate from which the Stage 8 fine-tuning pool is curated; the mutable record tracks quality and usage across cycles.

Field group	Contents
<i>Immutable content (never updated after storage)</i>	
Natural-language triple	question, reasoning chain, answer
Embedding	768-d L2-normalised SBERT vector (<code>float32</code>)
Provenance	storage cycle $\in \{0, \dots, N_{\text{cycles}}\}$, Unix timestamp
<i>Mutable quality record (updated by retroactive re-verification)</i>	
Nine signals	$u_{\text{token}}, u_{\text{dropout}}, u_{\text{internal}}, s_{\text{avg}}, h_{\text{norm}}, p_{\text{entail}}, p_{\text{ground}}^{\text{max}}, p_{\text{ground}}^{\text{mean}}, p_{\text{ground}}^{\text{atomic}}$
Veto signal	p_{contra} (triggers DISCARD on retroverify)
Composite	u_{stored} (Equation 1.6)
Decision record	one of STORE, DEFERRED, ABSTAIN, DISCARD; early-exit flag
<i>Mutable usage record (updated on each retrieval)</i>	
Retrieval count	integer r
Success rate	running fraction of retrievals accepted downstream
Retroverified flag	set at each cycle boundary; consumed by the Value score

product quantisation into $m = 64$ sub-vectors of 8 bits each, and search-time probing of $n_{\text{probe}} = 32$ centroids. This promotion is the operationalisation of Chapter 3’s memory-cost analysis: flat IP is cheap at $N \leq 20,000$ but scales poorly; IVF-PQ trades a small approximation error for sublinear retrieval and a $\sim 32\times$ reduction in embedding storage cost, which is what makes the $K_{\text{mem}} = 10^6$ target tractable on a single 24 GB device. Before promotion the quantiser is trained on a uniformly sampled subset of up to 200,000 existing vectors, which is sufficient because n_{list} is the binding constraint (FAISS requires at least as many training points as centroids, satisfied at the 20,000 threshold). After promotion the index returns approximate rather than exact nearest neighbours, but in practice the approximation does not change the routing decision: Stage 3’s tier boundaries at $S(q) \geq 0.90$ and $\text{sim}(q, e^*) \geq 0.75$ are far from the noise floor of IVF-PQ recall.

Per-query retrieval collapses to a single call: the query embedding $\hat{q}$ is submitted to the index, the top-1 neighbour e^* is returned with its cosine similarity and the external integer identifier under which its mutable record is kept in the metadata map. Stage 3 reads $u_{\text{stored}}(e^*)$ from the metadata map rather than from the index itself; storing composites inside the index is unnecessary because the identifier suffices and keeping quality metadata out of the FAISS payload avoids coupling the routing algorithm to the index’s internal layout.

Novelty filter. Before an episode is committed at Stage 7 it is screened for novelty: the memory is searched for its single nearest neighbour e^* , and storage is rejected when

$$\text{sim}(q, e^*) > \tau_{\text{nov}}, \quad \tau_{\text{nov}} = 0.95, \quad (1.8)$$

so that near-duplicates do not accumulate as separate entries. The filter is deliberately one-sided: a query sufficiently similar to an existing entry is assumed to be covered by that entry, and the existing entry’s mutable quality record is updated through the retrieval feedback loop rather than through a second store. The threshold is set high

(0.95) because SBERT similarities above this level are semantically indistinguishable in our pilot calibration set, while thresholds in the 0.85 to 0.90 range produced spurious rejections on paraphrases that legitimately tested a different aspect of the same topic. When a stored entry would otherwise be the k -th near-duplicate added, the training-pool curator of Stage 8 would be training many times on effectively the same (query, answer) pair, biasing the fine-tuning gradient; the novelty filter is the mechanism that prevents this.

Retrieval feedback update. Each accepted retrieval at Stage 4 triggers two updates on the retrieved entry’s mutable record. First, the retrieval count is incremented and the running success rate is updated by the standard incremental-mean formula,

$$r_{\text{new}} = r + 1, \quad \bar{a}_{\text{new}} = \frac{r \cdot \bar{a} + \mathbb{1}[\text{accepted}]}{r + 1}. \quad (1.9)$$

Second, once an entry has accumulated at least $r_{\text{min}} = 5$ retrievals, its stored composite is nudged by an exponential-moving-average rule with learning rate η ,

$$u_{\text{stored}} \leftarrow \begin{cases} u_{\text{stored}} + \eta(1 - u_{\text{stored}}) & \text{if the retrieval was accepted,} \\ u_{\text{stored}} - \eta u_{\text{stored}} & \text{otherwise,} \end{cases} \quad (1.10)$$

which draws the composite toward 1 on success and toward 0 on override. The floor of five retrievals before any update fires is important: the feedback rule is fundamentally noisy at low retrieval counts, and the cycle-boundary retroactive re-verification pass of Stage 8 is the authoritative quality update. The feedback rule therefore serves only as a fast-path adjustment between cycles, handling the case where an entry is retrieved heavily (and correctly or incorrectly) before the next cycle boundary arrives.

Capacity management by Value score. The store enforces a hard capacity bound $K_{\text{mem}} = 10^6$. When utilisation reaches a pruning trigger at 95% of capacity, the bottom 20% of entries by a Value score are removed and the FAISS index is rebuilt to reflect the deletions. The Value score is a convex combination of an Importance and a Recency component,

$$V(e) = w_{V,I} I(e) + w_{V,R} R(e), \quad w_{V,I} = 0.70, \quad w_{V,R} = 0.30, \quad (1.11)$$

with

$$I(e) = w_{I,\phi} \phi(e) + w_{I,r} \frac{r(e)}{r_{\text{max}}} + w_{I,s} \bar{a}(e) + w_{I,u} u_{\text{stored}}(e), \quad (1.12)$$

$$R(e) = \exp(-\lambda_R \text{age}(e)), \quad \lambda_R = 10^{-2}, \quad (1.13)$$

$$\phi(e) = (\text{cycle}(e) + 1) / (N_{\text{cycles}} + 1), \quad (1.14)$$

and the Importance weights $(w_{I,\phi}, w_{I,r}, w_{I,s}, w_{I,u}) = (0.40, 0.30, 0.20, 0.10)$. The cycle proxy $\phi(e)$ rewards entries stored by later, better-trained cycles; the normalisation $(N_{\text{cycles}} + 1)$ in the denominator guarantees $\phi \leq 1$ at the final cycle exactly rather than exceeding unity. The retrieval count enters only after normalisation by the global maximum r_{max} so that a single frequently-hit entry cannot dominate the ranking. The recency term uses real wall-clock age with $\lambda_R = 10^{-2}$ per second, a slow decay chosen so that within a single cycle’s duration the recency factor changes by at most a few percent; its role is to

tie-break between entries of similar importance rather than to dominate them. Pruning is upper-bounded in frequency by the trigger condition, and because every retained entry survives a convex-combination selection over four quality signals, the surviving population is on average better-calibrated than the pre-pruning population.

Persistence. The full store (FAISS index plus metadata map plus the monotonic next-identifier counter) is serialised to a pair of files at cycle boundaries. The index is written via FAISS’s native binary format; the metadata map is written as a pickle. On reload, the index’s IVF probing parameter is reset from the current configuration rather than from the saved index, so that n_{probe} can be tuned without rebuilding the memory. This is the mechanism by which a long-running experiment can be resumed at an arbitrary cycle boundary, and by which a retrieved store from one experimental configuration can be reused to seed another.

In summary, the memory module combines a normalised SBERT embedding space, a size-adaptive FAISS backend, a one-sided novelty filter, a per-retrieval feedback rule, and a Value-based pruning policy into a single persistent artefact whose query-time interface reduces to a single nearest-neighbour call plus a metadata lookup. All subsequent stages of CAEM are clients of this interface: Stage 3 consumes $(e^*, \text{sim}(q, e^*), u_{\text{stored}}(e^*))$ to route, Stage 7 calls the novelty filter and the add operation to write, and Stage 8 iterates over every stored entry to re-verify.

1.5 Three-Tier Router

Stage 3 converts the two upstream quantities (the pre-route confidence u_{pre} from Stage 1 and the single nearest-neighbour retrieval $(e^*, \text{sim}(q, e^*), u_{\text{stored}}(e^*))$ from Stage 2) into a categorical choice of processing tier. The tier choice determines the remainder of the per-query path: Tier 1 returns the stored answer attached to e^* without any generation, Tier 2 generates zero shot from the base model, and Tier 3 generates under retrieval-augmented grounding against a preprocessed Wikipedia passage index. The router is the cost-control mechanism of the pipeline: every decision it makes to dispatch to Tier 1 rather than Tier 2 or 3 saves a decoder pass, and every decision it makes to dispatch to Tier 3 rather than Tier 2 incurs passage retrieval and reranking but buys grounding that the verifier can subsequently exploit. The design of the router therefore has to trade cost against safety while keeping the trade explicit.

Two-mechanism design. The router applies two decision rules that are deliberately kept separate. The first rule is a safety override evaluated before any score is computed: if u_{pre} falls below a minimum floor $\tau_{\text{safe}} = 0.60$, the query is dispatched to Tier 3 regardless of how well it matched the memory. The second rule is a combined routing score over similarity and stored composite, thresholded to select between Tier 1 and Tier 2. The two rules are not merged into a single formula. The reason is arithmetic: if the safety condition entered additively alongside similarity and u_{stored} , a high stored composite from a very similar episode could arithmetically compensate for a dangerously low u_{pre} , sending the query to Tier 1 exactly when the base model’s internal state warns that it should not be trusted. Treating model readiness (u_{pre}) and memory quality (u_{stored}) as two independent necessary conditions rather than as tradeable summands preserves the semantics that a low u_{pre} is a veto on the cheap paths, not a discount to be offset.

Safety override. The first mechanism, evaluated before any score calculation, is the OR-condition:

$$u_{\text{pre}} < \tau_{\text{safe}} \quad \Rightarrow \quad \text{Tier 3}, \quad (1.15)$$

with $\tau_{\text{safe}} = 0.60$. Three observations motivate the floor value. First, u_{pre} is temperature-calibrated from Cycle 1 onward (Section 1.3), so the threshold is referenced to a distribution of confidences that has been brought into alignment with empirical exact-match correctness; a floor of 0.60 corresponds in that calibrated regime to roughly a 40% confident-error risk on the cheap paths, which is unacceptable in an open-domain factual setting. Second, Tier 3 is the only path that introduces external evidence into the verifier, so forcing grounding is the operationally meaningful response to low model readiness: the verifier’s grounding family $\{p_{\text{ground}}^{\text{max}}, p_{\text{ground}}^{\text{mean}}, p_{\text{ground}}^{\text{atomic}}\}$ cannot fire on a Tier 1 or Tier 2 answer because no passages are retrieved. Third, the safety override sets a cost floor on queries the system distrusts: a Tier 3 pass takes a few seconds while a Tier 1 hallucination would propagate into memory (through Stages 5 and 7) and potentially into the next cycle’s fine-tuning pool. The cost asymmetry makes Tier 3 the unambiguously correct default when the model itself indicates uncertainty.

Combined routing score. When the safety override does not fire, the router evaluates a combined score over the similarity and the stored composite of the retrieved episode,

$$S(q) = \alpha \text{sim}(q, e^*) + (1 - \alpha) u_{\text{stored}}(e^*), \quad \alpha = 0.70, \quad (1.16)$$

so that similarity carries more weight than the stored composite by a ratio of 7:3. The weighting is deliberate. Similarity is the direct relevance signal: it measures whether the stored episode is about the same question as the query, and no Tier 1 outcome is defensible without high similarity. The stored composite is an orthogonal quality signal that reflects how well the stored answer was verified at storage time and subsequently, but a very high u_{stored} on a loosely related episode is not an argument for returning that episode’s answer; it is an argument for not using Tier 1 at all. Up-weighting similarity therefore encodes the operational priority (relevance first, quality as a secondary gate), while retaining the stored composite in the score means that two episodes at equal similarity are separated by their verification history, with better-verified ones clearing the Tier 1 threshold sooner.

Tier dispatch rules. Given the combined score $S(q)$ and the raw similarity $\text{sim}(q, e^*)$, the router selects a tier according to Table 1.3. Tier 1 requires the combined score to clear $\tau_{\text{tier1}} = 0.90$, which is tight enough that Tier 1 is selected only when both signals are strong: at the limit, a similarity of 1.00 with an u_{stored} of 0.77 clears the threshold, and so does a similarity of 0.95 paired with u_{stored} of 0.78. Tier 2 is selected when Tier 1 is not taken and the raw similarity alone exceeds $\tau_{\text{tier2}} = 0.75$; this captures the case where an episode is clearly topically relevant but the composite alone is not yet strong enough for cheap-path reuse, and zero-shot generation is expected to produce a defensible answer that the verifier can score. Tier 3 is selected in every remaining case: empty memory, low similarity, or borderline combined scores that do not clear the Tier 1 bar without also having the topical similarity to justify Tier 2.

Behaviour on an empty memory. The router requires no special case for the cold-start regime in which the episodic store is empty. On an empty memory Stage 2 returns an empty result, which the router interprets by substituting $\text{sim}(q, e^*) = 0$ and $u_{\text{stored}}(e^*) = 0$

Table 1.3: Tier dispatch conditions at Stage 3. Conditions are evaluated in order. The safety override supersedes the combined-score rules; when it does not fire, the combined score selects Tier 1 or the similarity selects Tier 2, and Tier 3 is the default.

Tier	Condition (evaluated in order)	Action
3 (forced)	$u_{\text{pre}} < \tau_{\text{safe}} = 0.60$ (safety override)	Forced RAG; skips score.
1	$S(q) \geq \tau_{\text{tier1}} = 0.90$ and safety override not taken	Return e^* 's stored answer.
2	$\text{sim}(q, e^*) > \tau_{\text{tier2}} = 0.75$, Tier 1 not taken	Zero-shot generation.
3 (default)	otherwise (including empty memory)	DPR retrieve + rerank + RAG.

into Equation 1.16, giving $S(q) = 0$. The combined score then falls below both tier thresholds, the similarity fails the Tier 2 bar, and the query dispatches to Tier 3 by default. The safety override rule is unaffected: it depends only on u_{pre} , which is produced by Stage 1 regardless of memory state. As a consequence, the entire first cycle of CAEM (Cycle 0, during which the memory is initialised and the calibration split is collected) runs exclusively on Tiers 2 and 3, which is the intended behaviour: the cheap Tier 1 path comes online organically as u_{stored} values accumulate at Stage 7.

Algorithm 1.2 states the Stage 3 logic in its entirety. The strict evaluation order (override first, then score) is load-bearing and is the algorithmic encoding of the two-mechanism separation argued above.

Algorithm 1.2 Stage 3 three-tier routing decision.

Require: Pre-route confidence u_{pre} ; retrieved neighbour $(e^*, \text{sim}(q, e^*), u_{\text{stored}}(e^*))$ or empty; weight $\alpha = 0.70$; thresholds $\tau_{\text{safe}} = 0.60$, $\tau_{\text{tier1}} = 0.90$, $\tau_{\text{tier2}} = 0.75$.

Ensure: Tier $t \in \{1, 2, 3\}$ with a safety-override flag.

```

1: if memory returned no result then
2:    $\text{sim}(q, e^*) \leftarrow 0$ ;  $u_{\text{stored}}(e^*) \leftarrow 0$                                  $\triangleright$  empty-memory default
3: end if
4: if  $u_{\text{pre}} < \tau_{\text{safe}}$  then
5:   return ( $t = 3$ , safety_override = true)                                 $\triangleright$  mechanism 1: OR-condition
6: end if
7:  $S(q) \leftarrow \alpha \text{sim}(q, e^*) + (1 - \alpha) u_{\text{stored}}(e^*)$                  $\triangleright$  mechanism 2: combined score
8: if  $S(q) \geq \tau_{\text{tier1}}$  then
9:   return ( $t = 1$ , safety_override = false)
10: else if  $\text{sim}(q, e^*) > \tau_{\text{tier2}}$  then
11:   return ( $t = 2$ , safety_override = false)
12: else
13:   return ( $t = 3$ , safety_override = false)
14: end if
```

The routing decision is logged on a per-query basis with all five upstream quantities (u_{pre} , $\text{sim}(q, e^*)$, $u_{\text{stored}}(e^*)$, $S(q)$, and the safety-override flag), because these form the substrate of the routing-distribution analyses reported in Chapter 5. In particular, the fraction of queries that triggered the safety override separately from the fraction that were

routed to Tier 3 by the default rule are tracked as distinct metrics: the two populations differ in whether the model *recognised* that it was uncertain (override) versus the memory simply not containing a relevant neighbour (default), and both are reported in the routing-audit table of Chapter 5.

1.6 Unified Verifier

Every Tier 2 and Tier 3 answer passes through a single Stage 5 quality gate before the storage decision is made. The verifier consumes a (query, answer) pair and emits nine scalar signals together with a composite score u_{stored} and an explicit flag for a confabulation early-exit gate. The signal set is designed around a single organising claim: any single confidence signal is exploitable in one direction or another, so the composite is built from three complementary families that fail in different ways. Internal calibration signals are cheap and fine-grained but do not know whether the answer is true; sample-set signals detect disagreement across the model’s own resamplings but cannot distinguish a confidently-wrong consensus from a correct one; external grounding signals test factual support against retrieved evidence but are vulnerable to partially-correct answers that are superficially supported. The composite combines all three so that no single failure mode clears the storage threshold on its own. This section specifies the nine signals, their aggregation rules, and the early-exit gate. The four-outcome decision tree that consumes the composite is the subject of Section 1.7.

Signal families and cost. Table 1.4 summarises all nine signals together with their source family, the number of model calls each contributes, and whether the signal is literature-derived (L) or a CAEM-specific design decision (D). The per-query cost of Stage 5 is dominated by the decoding passes required for the sample-set family: $M = 3$ chains at temperature $T_s = 0.7$ for s_{avg} and p_{entail} (shared), $K = 10$ independent samples at temperature $T_{\text{se}} = 1.0$ for h_{norm} , and $K_D = 5$ stochastic passes for u_{dropout} . The external-grounding family adds 20 passages retrieved from the preprocessed Wikipedia index and reranked to the top 3 by a cross-encoder, plus one NLI ensemble call per passage. The atomic-fact path additionally performs one greedy decomposition pass and one NLI call per atom per passage. All nine signals are computed once per Stage 5 invocation; there is no adaptive early stopping inside the signal computation itself (the only early exit is the composite-level confabulation gate described below).

Internal calibration. The internal family measures the model’s self-assessed confidence in the specific answer under evaluation, using no external information. The token aggregate u_{token} here is defined analogously to Stage 1’s (Equation 1.1) but is computed over the full emitted answer rather than over the greedy prefix and is evaluated under teacher forcing with the answer tokens as labels, so it measures the model’s probability mass on the exact string being verified:

$$u_{\text{token}}^{\text{verify}} = \exp\left(\frac{1}{n} \sum_{i=1}^n \log p_{\theta}(y_i \mid y_{<i}, q)\right), \quad (1.17)$$

with non-padding positions only contributing to the mean. The dropout signal follows the predictive-uncertainty framing of Gal and Ghahramani [?]: dropout is left active during

Table 1.4: The nine verifier signals, grouped by family. Source column: [L] = adapted from literature, [D] = CAEM design. Cost column reports the dominant factor per query at Stage 5.

Signal	Family / what it measures	Source	Dominant cost
u_{token}	Internal: lexical confidence in the answer	[L]	1 teacher-forced forward pass
u_{dropout}	Internal: MC-dropout disagreement across stochastic passes	[L]	$K_D = 5$ decoding passes
u_{internal}	Internal: fused scalar $0.5 u_{\text{token}} + 0.5 (1 - u_{\text{dropout}})$	[D]	(reuses above)
s_{avg}	Sample-set: mean pairwise SBERT cosine across M chains	[L]	$M = 3$ decoding passes (shared)
p_{entail}	Sample-set: NLI entailment chain-to-answer, averaged over M chains	[D]	M NLI calls (reuses chains)
h_{norm}	Sample-set: normalised semantic entropy over K samples, NLI-clustered	[L]	$K = 10$ decoding passes
$p_{\text{ground}}^{\text{max}}$	Grounding: max NLI entailment across top-3 reranked passages	[D]	3 NLI calls
$p_{\text{ground}}^{\text{mean}}$	Grounding: mean NLI entailment across top-3 reranked passages	[D]	(reuses above)
$p_{\text{ground}}^{\text{atomic}}$	Grounding: weakest-link entailment over decomposed atomic facts	[D]	n_{atoms} NLI calls plus 1 decode
p_{contra}	Veto signal: max contradiction probability over top-3 passages	[D]	3 NLI calls (reuses ensemble)

inference and the generator is sampled $K_D = 5$ times at $T = 0.7$; a plurality answer is identified over the K_D samples, and

$$u_{\text{dropout}} = 1 - \frac{\max_c \#\{i : a_i \equiv a_c\}}{K_D} \in [0, 1 - 1/K_D], \quad (1.18)$$

where $a_i \equiv a_c$ denotes surface-normalised string equality. Higher values indicate that the model disagrees with itself under perturbation, and the composite uses $(1 - u_{\text{dropout}})$ through the fused internal scalar. The fused internal scalar averages the two,

$$u_{\text{internal}} = \frac{1}{2} u_{\text{token}}^{\text{verify}} + \frac{1}{2} (1 - u_{\text{dropout}}), \quad (1.19)$$

with equal weighting because the two signals capture qualitatively different failure modes: $u_{\text{token}}^{\text{verify}}$ is degraded by any token the model considers unlikely even when the overall answer is correct, and u_{dropout} is degraded only when the model actively disagrees with itself; pairing them tightens the internal signal against either individual noise source. The fused scalar is the signal that enters the composite directly, and is also the signal checked by the confabulation gate.

Sample-set signals. The sample-set family examines what the model produces when asked the same query multiple times with non-deterministic sampling. Three signals share this family. For the first two, $M = 3$ chain-of-thought responses are generated at $T = 0.7$ and reused by both signals. The self-consistency signal s_{avg} is the mean pairwise cosine similarity between chain embeddings produced by the same SBERT model used by the memory (Equation 1.7),

$$s_{\text{avg}} = \binom{M}{2}^{-1} \sum_{1 \leq i < j \leq M} \hat{c}_i^\top \hat{c}_j, \quad (1.20)$$

with the unique-pair $i < j$ convention so the diagonal and double counting are not included; this is the formulation of Wang et al. [?]. The chain-to-answer entailment signal is the mean over the same M chains of the NLI ensemble’s entailment probability from chain to answer,

$$p_{\text{entail}} = \frac{1}{M} \sum_{i=1}^M \text{NLI}_{\min}(c_i \Rightarrow a), \quad (1.21)$$

where $\text{NLI}_{\min}$ denotes the minimum entailment probability across all NLI bundles in the ensemble (the conservative aggregation rule described below). The two signals combine differently: s_{avg} rewards surface and semantic consistency across the chain space, while p_{entail} rewards chains that individually support the final answer under entailment, so they decorrelate on cases where the model produces several internally-consistent but collectively-wrong chains.

The third sample-set signal is the normalised semantic entropy of Farquhar et al. [?], computed over an independent set of $K = 10$ samples at $T = 1.0$. Samples are clustered by bidirectional NLI entailment: two samples s_i and s_j join the same cluster iff

$$\text{NLI}(s_i \Rightarrow s_j) = \text{ENT} \wedge \text{NLI}(s_j \Rightarrow s_i) = \text{ENT}, \quad (1.22)$$

where ENT is the argmax-label under the NLI ensemble’s majority-vote rule. The cluster assignment induces a distribution over cluster sizes, from which the Shannon entropy is computed and normalised by $\log_2 K$,

$$h_{\text{norm}} = \frac{-\sum_c p_c \log_2 p_c}{\log_2 K}, \quad p_c = \frac{|\{i : s_i \in c\}|}{K}, \quad (1.23)$$

so that $h_{\text{norm}} \in [0, 1]$. A value near zero indicates that almost all samples express semantically equivalent answers, while a value near one indicates that the samples partition into approximately K distinct semantic classes. The composite treats $(1 - h_{\text{norm}})$ as a confidence: semantic agreement across resamples is a positive signal. When no NLI ensemble is configured, the fallback is surface-level entropy over normalised strings, which is strictly weaker because it cannot merge paraphrases; this fallback matters only in smoke-test configurations.

External grounding. The grounding family anchors the verifier to external evidence rather than to the model’s own output distribution. On every Tier 2 and Tier 3 verification, the preprocessed Wikipedia passage retriever is called with the query and asked for the top $K_{\text{retrieve}} = 20$ candidates; these are then reranked by a cross-encoder scored against the concatenated (query, answer) pair, and the top $K_{\text{rerank}} = 3$ are retained. Two NLI-based signals are computed on this top-3 set:

$$p_{\text{ground}}^{\text{max}} = \max_{p \in P_{\text{top3}}} \text{NLI}_{\text{min}}(p \Rightarrow a), \quad (1.24)$$

$$p_{\text{ground}}^{\text{mean}} = \frac{1}{|P_{\text{top3}}|} \sum_{p \in P_{\text{top3}}} \text{NLI}_{\text{min}}(p \Rightarrow a), \quad (1.25)$$

together with the contradiction veto

$$p_{\text{contra}} = \max_{p \in P_{\text{top3}}} \text{NLI}_{\text{max}}^{\text{contra}}(p \Rightarrow a), \quad (1.26)$$

where $\text{NLI}_{\text{max}}^{\text{contra}}$ denotes the maximum contradiction probability across NLI bundles (the safety aggregation described below). The max and mean aggregates have complementary failure modes. $p_{\text{ground}}^{\text{max}}$ is robust to a partially-incorrect passage set because a single strongly-entailing passage suffices, but it is vulnerable to a superficially-matching passage that happens to mention the answer tokens without actually supporting the claim. $p_{\text{ground}}^{\text{mean}}$ is the reverse: less vulnerable to a single false-match passage, more sensitive to the quality of retrieval as a whole. Both enter the composite so that the resulting estimate is positive only when evidence is both present (max is high) and broadly supportive (mean is high).

Atomic-fact decomposition. The atomic signal is the one grounding mechanism that is not available through a single answer-level NLI call. It addresses a specific failure mode the max and mean aggregates cannot distinguish: an answer composed of several factual claims of which one is supported by the passages and another is silently fabricated, where the passage-level entailment is high enough on the supported claim to dominate the aggregates. The mechanism proceeds in three steps. First, the base model is prompted with a fixed instruction to decompose the answer into a numbered list of independent factual claims with pronouns resolved,

$$(a_1, \dots, a_{n_{\text{atoms}}}) = \text{Decompose}(a), \quad (1.27)$$

and the numbered output is parsed into a deduplicated list of claims. Second, each claim is individually tested against each of the top-3 reranked passages via the NLI ensemble’s max-entail rule, giving a per-claim grounding score

$$g_k = \max_{p \in P_{\text{top3}}} \text{NLI}_{\text{min}}(p \Rightarrow a_k), \quad k = 1, \dots, n_{\text{atoms}}. \quad (1.28)$$

Third, the atomic signal is defined as the weakest link across claims,

$$p_{\text{ground}}^{\text{atomic}} = \min_k g_k, \quad (1.29)$$

so that the addition of a single unsupported claim suppresses the signal. When decomposition fails (parsing error, empty list, or the atomic path is disabled in configuration), the atomic signal falls back to $p_{\text{ground}}^{\text{mean}}$ rather than to zero; this prevents the composite from silently penalising answers whose decomposition simply happened to fail for tokenisation reasons, while ensuring that atomic decomposition adds information when it succeeds.

NLI ensemble aggregation. Every NLI-based signal in this section is computed over an ensemble of two pretrained MNLI models (RoBERTa-Large-MNLI and DeBERTa-v3-Large-MNLI), with two aggregation rules chosen to be consistently conservative. Entailment probabilities are aggregated by minimum across the ensemble (NLI_{min}): both models must agree that a premise entails a hypothesis before the signal is allowed to be high. Contradiction probabilities are aggregated by maximum ($\text{NLI}_{\text{max}}^{\text{contra}}$): any model flagging a contradiction is enough to raise the veto. The rule is asymmetric because it consistently biases toward not storing a verified answer when the ensemble disagrees, which is the direction the composite is defended against. A single-model fallback is supported in configurations where only one NLI bundle is available, in which case the min and max reduce to that bundle’s own probability; no fallback signal is returned in the zero-model case, and the verifier’s output on those signals defaults to 0.5 for entailment and 0.0 for contradiction so that the composite neither rewards nor punishes in the absence of evidence.

Composite score. The six non-redundant signals from the nine (the three internal collapse to u_{internal} , the three sample-set contribute separately, the three grounding contribute separately) are combined into the composite u_{stored} of Equation 1.6, reproduced here in the form the verifier actually evaluates:

$$u_{\text{stored}} = w_{\text{g,mean}} p_{\text{ground}}^{\text{mean}} + w_{\text{g,atom}} p_{\text{ground}}^{\text{atomic}} + w_{\text{entail}} p_{\text{entail}} + w_{\text{sc}} s_{\text{avg}} + w_{\text{int}} u_{\text{internal}} + w_{\text{se}} (1 - h_{\text{norm}}), \quad (1.30)$$

with weights $(w_{\text{g,mean}}, w_{\text{g,atom}}, w_{\text{entail}}, w_{\text{sc}}, w_{\text{int}}, w_{\text{se}}) = (0.30, 0.15, 0.15, 0.15, 0.15, 0.10)$ summing to unity so the composite remains in $[0, 1]$. The weight allocation is a deliberate reflection of the signal-family design. The largest single weight (0.30) is placed on $p_{\text{ground}}^{\text{mean}}$ because broad passage-level entailment is the most direct test of factual correctness; the four equal weights at 0.15 on $p_{\text{ground}}^{\text{atomic}}$, p_{entail} , s_{avg} , and u_{internal} reflect a deliberately diffuse prior over the secondary signals, with a single joint ablation of the weights reported in Chapter 5 (ablation variant **equal_signal_weights**) testing whether the split is genuinely informative; and the smallest weight (0.10) on $(1 - h_{\text{norm}})$ reflects the intermediate position of semantic entropy, which is informative but noisiest among the six because it is sensitive to cluster-count artefacts at small K . The contradiction probability p_{contra} does not enter the composite because it operates as a veto consumed by the decision tree of Section 1.7 rather than as a smooth signal: averaging a contradiction into the composite would let a strong entailment arithmetically outweigh a contradiction, which is exactly the failure mode the decision tree is designed to reject categorically.

Confabulation early-exit gate. Before the composite is formed, the verifier evaluates one hard safety check:

$$u_{\text{internal}} \geq \tau_{\text{ee,int}} \wedge p_{\text{ground}}^{\text{max}} \leq \tau_{\text{ee,g}} \Rightarrow \text{early exit, DISCARD}, \quad (1.31)$$

with $\tau_{\text{ee,int}} = 0.70$ and $\tau_{\text{ee,g}} = 0.20$. The rule targets the specific failure mode where the base model is highly self-confident in an answer that has essentially no external grounding: not only does no retrieved passage broadly support the answer (this is what $p_{\text{ground}}^{\text{mean}}$ would catch), but not even the single most-supportive passage in the top three reaches a low entailment threshold of 0.20. When both conditions hold, the composite is not even computed, the outcome is set to DISCARD with the early-exit flag set, and all nine signals are still recorded for analysis. The gate is conservative: it fires only when internal confidence is high and grounding is effectively absent, which is exactly the region of the signal space where the composite could otherwise be raised by the sample-set signals into a STORE-range value on an answer the ensemble of NLI models cannot find support for. The precise thresholds were selected so that the gate fires on a manageable minority of Tier 2 and Tier 3 outputs at Cycle 0 and is tracked across cycles as a diagnostic in Chapter 5: a cycle-over-cycle rise would indicate that fine-tuning is introducing confabulations rather than resolving them.

Algorithm 1.3 specifies the complete verifier, terminating with the composite u_{stored} and the early-exit flag; the four-outcome decision tree that consumes these values is described in Section 1.7.

Algorithm 1.3 Stage 5 unified verifier.

Require: Query q ; answer a ; base model f_θ ; SBERT encoder; NLI ensemble; passage retriever and reranker; composite weights as in Equation 1.30.

Ensure: Nine signals, composite u_{stored} , early-exit flag.

- 1: $u_{\text{token}}^{\text{verify}} \leftarrow$ teacher-forced geometric mean log-prob of a under $f_\theta(q)$
 - 2: $\{a_i\}_{i=1}^{K_D} \leftarrow$ generate $K_D = 5$ samples with dropout active at $T = 0.7$
 - 3: $u_{\text{dropout}} \leftarrow 1 - \max_c \#\{i : a_i \equiv a_c\} / K_D$
 - 4: $u_{\text{internal}} \leftarrow \frac{1}{2} u_{\text{token}}^{\text{verify}} + \frac{1}{2} (1 - u_{\text{dropout}})$
 - 5: $\{c_i\}_{i=1}^M \leftarrow$ generate $M = 3$ chains at $T = 0.7$
 - 6: $s_{\text{avg}} \leftarrow \binom{M}{2}^{-1} \sum_{i < j} \hat{c}_i^\top \hat{c}_j$
 - 7: $p_{\text{entail}} \leftarrow \frac{1}{M} \sum_i \text{NLI}_{\min}(c_i \Rightarrow a)$
 - 8: $\{s_j\}_{j=1}^K \leftarrow$ generate $K = 10$ samples at $T = 1.0$
 - 9: cluster $\{s_j\}$ by bidirectional NLI entailment; $h_{\text{norm}} \leftarrow -\sum_c p_c \log_2 p_c / \log_2 K$
 - 10: $P_{\text{top20}} \leftarrow \text{Retrieve}(q, K_{\text{retrieve}} = 20)$; $P_{\text{top3}} \leftarrow \text{Rerank}(q, a, P_{\text{top20}}, K_{\text{rerank}} = 3)$
 - 11: $p_{\text{ground}}^{\text{max}}, p_{\text{ground}}^{\text{mean}} \leftarrow$ max and mean of $\text{NLI}_{\min}(p \Rightarrow a)$ over P_{top3}
 - 12: $p_{\text{contra}} \leftarrow \max_{p \in P_{\text{top3}}} \text{NLI}_{\max}^{\text{contra}}(p \Rightarrow a)$
 - 13: $(a_1, \dots, a_{n_{\text{atoms}}}) \leftarrow \text{Decompose}(a)$ $\triangleright$ fallback to $p_{\text{ground}}^{\text{mean}}$ on failure
 - 14: $p_{\text{ground}}^{\text{atomic}} \leftarrow \min_k \max_{p \in P_{\text{top3}}} \text{NLI}_{\min}(p \Rightarrow a_k)$
 - 15: **if** $u_{\text{internal}} \geq \tau_{\text{ee,int}}$ **and** $p_{\text{ground}}^{\text{max}} \leq \tau_{\text{ee,g}}$ **then**
 - 16: **return** all signals, $u_{\text{stored}} \leftarrow 0$, early-exit = true $\triangleright$ confabulation gate
 - 17: **end if**
 - 18: $u_{\text{stored}} \leftarrow$ Equation 1.30
 - 19: **return** all signals, u_{stored} , early-exit = false
-

The verifier is the most expensive per-query computation in CAEM (several decoder invocations, ensemble NLI calls over up to four premise-hypothesis pairs, one cross-encoder

rerank, and a bounded number of additional NLI calls in the atomic path), which is why only Tier 2 and Tier 3 queries invoke it. Tier 1 queries are served from memory under the invariant that every stored entry cleared this verifier at its original storage event and has been re-verified at every cycle boundary since (Section 1.8). The composite u_{stored} and the veto signal p_{contra} are the two quantities consumed by the Stage 6 decision tree; the remaining eight signals are recorded on the episode for downstream calibration, ablation, and retroactive re-verification.

1.7 Four-Outcome Storage Decision

Stage 6 consumes three scalars from the verifier (composite u_{stored} , contradiction probability p_{contra} , and the max grounding probability $p_{\text{ground}}^{\text{max}}$) and emits one of four categorical outcomes: STORE, DEFERRED, ABSTAIN, or DISCARD. Stage 7 is the mechanical consumer of that decision: it writes the episode to memory, withholds it from the training pool, returns a principled refusal, or suppresses the generation entirely. The two stages are treated together in this section because they form a single policy surface: the decision tree is meaningful only through the actions it triggers.

Most verification pipelines collapse Stage 6 into a binary store-or-discard rule on a thresholded composite. CAEM retains four outcomes because the composite alone is not sufficient to distinguish between confidently-wrong answers that must be suppressed, uncertain answers that the system should refuse rather than answer, borderline answers that merit another look at the next cycle boundary, and high-quality answers that are safe to commit. Collapsing the middle cases into either STORE or DISCARD either pollutes the Tier 1 path with marginal answers or discards recoverable signal that the retroactive re-verification pass (Section 1.8) would otherwise promote. The four-outcome structure is therefore a recoverability argument: Stage 6 allows a third state (DEFERRED) in which an answer’s quality is good enough to keep around but not good enough to act on, and a fourth state (ABSTAIN) in which the system owns up to uncertainty rather than silently storing or silently discarding.

Contradiction veto. The decision tree begins with a hard veto on contradiction, evaluated before any composite-threshold check:

$$p_{\text{contra}} \geq \tau_{\text{veto}} \quad \Rightarrow \quad \text{DISCARD}, \quad \tau_{\text{veto}} = 0.30. \quad (1.32)$$

The veto is categorical because contradiction is an asymmetric signal: a passage that contradicts the answer is evidence of incorrectness that cannot be arithmetically outweighed by even a very high composite. If a strong entailment on one passage and a strong contradiction on another both enter a weighted average, a high average implies nothing about whether the answer is true. Stage 6 therefore removes p_{contra} from the composite altogether (Section 1.6) and consumes it only as a veto at this step. The threshold $\tau_{\text{veto}} = 0.30$ is deliberately low: an ensemble-max contradiction probability of 0.30 means that at least one NLI bundle assigns 30% probability to a contradiction claim on at least one of the top-three passages. Because of the asymmetric ensemble aggregation (max rather than min for contradiction), this corresponds to a very weak threshold on single-model contradiction detection, which is appropriate for a veto that is supposed to be a tripwire rather than a smooth signal.

Store and Deferred thresholds. When the veto does not fire, the composite u_{stored} is thresholded against two cutoffs in a cascade:

$$\text{STORE if } u_{\text{stored}} \geq \tau_{\text{store}}, \quad \text{DEFERRED if } \tau_{\text{defer}} \leq u_{\text{stored}} < \tau_{\text{store}}, \quad (1.33)$$

with $\tau_{\text{store}} = 0.65$ and $\tau_{\text{defer}} = 0.45$. The two thresholds partition the composite range into three bands. The upper band ($u_{\text{stored}} \geq 0.65$) authorises a commit: the composite is high enough that the answer is treated as verified knowledge, with all the downstream consequences (memory write, eligibility for the training pool, availability for Tier 1 retrieval on future queries). The middle band ($0.45 \leq u_{\text{stored}} < 0.65$) authorises a hold: the answer is plausibly correct (it cleared the 0.45 floor, which is already well above a confidently-wrong answer’s composite) but has not cleared the commit bar. The lower band falls through to the remaining checks. The gap between the two thresholds, and its midpoint value of 0.55, is the single tuning knob with the most influence on the STORE-to-DEFERRED ratio across the evaluation corpus; a tighter gap would collapse DEFERRED into STORE at the risk of training on under-verified material, and a wider gap would shift recoverable answers into the DISCARD state and lose them permanently.

Abstain rule. Below τ_{defer} , the decision tree evaluates a grounding-floor check before falling through to DISCARD:

$$u_{\text{stored}} < \tau_{\text{defer}} \wedge p_{\text{ground}}^{\text{max}} < \tau_{\text{abs}} \quad \Rightarrow \quad \text{ABSTAIN}, \quad \tau_{\text{abs}} = 0.20. \quad (1.34)$$

The rule identifies the specific regime in which the composite is low and the single most-supportive retrieved passage does not meaningfully support the answer: there is neither internal confidence that the answer is right nor external evidence that it is right. In this regime, ABSTAIN is preferred over DISCARD because the two outcomes differ in what the pipeline returns to the user. A DISCARD suppresses the generated answer and emits nothing downstream-usable; an ABSTAIN returns a principled refusal (an explicit “I do not know” response), which is a useful output in factual QA settings where the failure mode of highest cost is a confident wrong answer rather than a refusal. The grounding-floor threshold $\tau_{\text{abs}} = 0.20$ is intentionally low: it fires only when the top-3 passages collectively fail to produce even a weakly-supporting entailment, which is the signal that no recoverable evidence exists in the current retrieval neighbourhood.

Discard as the default. Every case that does not match the preceding rules falls through to DISCARD. There are two distinct scenarios in which the default fires. The first is the contradiction-veto scenario of Equation 1.32, evaluated first. The second is the intermediate scenario in which the composite is low ($u_{\text{stored}} < \tau_{\text{defer}}$) but retrieval did produce at least one supporting passage ($p_{\text{ground}}^{\text{max}} \geq \tau_{\text{abs}}$): in this case the answer is not good enough to keep, but the presence of some grounding signal means the failure was not of the recoverable-refusal kind (the query did have evidence in the retrieval index; the model simply failed to use it). Reporting a DISCARD rather than an ABSTAIN in this case is the decision tree’s way of distinguishing “we had evidence and still got it wrong” from “we had no evidence at all”, because the two motivate different downstream responses in Chapter 5’s error analysis: the first is evidence of a generation failure, the second of a retrieval failure.

Table 1.5: The four-outcome decision tree at Stage 6 and the Stage 7 action it triggers. Conditions are evaluated in order; the first matching condition determines the outcome.

Outcome	Condition (in order)	Stage 7 action
DISCARD (veto)	$p_{\text{contra}} \geq \tau_{\text{veto}} = 0.30$	Suppress output; no memory write.
STORE	$u_{\text{stored}} \geq \tau_{\text{store}} = 0.65$	Write episode; eligible for training if $u_{\text{stored}} \geq \tau_{\text{train}} = 0.75$.
DEFERRED	$\tau_{\text{defer}} \leq u_{\text{stored}} < \tau_{\text{store}}$, i.e. $0.45 \leq u_{\text{stored}} < 0.65$	Retained for cycle-boundary re-evaluation; not used for training at the current cycle.
ABSTAIN	$u_{\text{stored}} < \tau_{\text{defer}}$ and $p_{\text{ground}}^{\text{max}} < \tau_{\text{abs}} = 0.20$	Return principled refusal to user; no memory write.
DISCARD (default)	otherwise	Suppress output; no memory write.

Stage 7 consequences and training eligibility. Stage 7 is deterministic given the decision: STORE writes a complete EPISODICENTRY (Table 1.2) with the full nine-signal record, the composite, the contradiction probability, the early-exit flag, and the decision string itself; DEFERRED retains the episode for re-consideration at the next cycle boundary without committing it as a first-class memory entry; ABSTAIN returns a refusal to the user; DISCARD suppresses the generation. Two additional invariants apply at write time. First, a STORE decision is subject to the novelty filter of Equation 1.8: if the query is too similar to an existing memory entry, the store is rejected at Stage 7 even though Stage 6 green-lit it, and the existing entry’s retrieval feedback fields are updated instead. Second, an entry is flagged as eligible for the fine-tuning pool only if its composite further clears a training-eligibility threshold $\tau_{\text{train}} = 0.75$; this is a stricter bar than the STORE threshold because the training pool must be composed of only the most well-verified material, while the Tier 1 retrieval path can safely serve from a broader band of quality. The gap between $\tau_{\text{store}} = 0.65$ and $\tau_{\text{train}} = 0.75$ is therefore an architectural commitment: Tier 1 retrieval and training data curation have different quality bars, and conflating them would either starve the training pool or over-serve Tier 1 from marginal memory.

Algorithm 1.4 states the Stage 6 and Stage 7 logic in the evaluation order the code uses. The order is load-bearing: the contradiction veto precedes the composite threshold so that a high-contradiction answer with a high composite is discarded rather than stored; the ABSTAIN check precedes the default DISCARD so that the refusal path is taken whenever grounding is absent rather than merely weak; and the training-eligibility check is applied only after the memory write so that a deferred entry does not accidentally enter the pool.

The per-cycle distribution over the four outcomes is one of the principal diagnostics reported in Chapter 5, because it reveals how the verifier is carving up the generation stream and how this carving evolves across cycles: a healthy trajectory shifts mass from DEFERRED into STORE as the base model fine-tunes on the training pool (answers the model was borderline about in cycle n become commits in cycle $n+1$), while a pathological trajectory shifts mass from STORE into DISCARD (the model is drifting into confabulation and the early-exit gate plus contradiction veto are increasingly firing). The categorical structure of the Stage 6 decision therefore is not only a quality control mechanism at the per-query level but a diagnostic at the cycle level, and Chapter 5 reports the full (cycle

Algorithm 1.4 Stage 6 decision tree and Stage 7 action.

Require: Composite u_{stored} ; contradiction p_{contra} ; max grounding $p_{\text{ground}}^{\text{max}}$; thresholds $\tau_{\text{veto}} = 0.30$, $\tau_{\text{store}} = 0.65$, $\tau_{\text{defer}} = 0.45$, $\tau_{\text{abs}} = 0.20$, $\tau_{\text{train}} = 0.75$.

Ensure: Outcome $\in \{\text{STORE}, \text{DEFERRED}, \text{ABSTAIN}, \text{DISCARD}\}$; Stage 7 action.

```

1: if  $p_{\text{contra}} \geq \tau_{\text{veto}}$  then
2:   return DISCARD; suppress output
3: end if
4: if  $u_{\text{stored}} \geq \tau_{\text{store}}$  then
5:   if novelty filter passes (Eq. 1.8) then write episode to memory
6:   if  $u_{\text{stored}} \geq \tau_{\text{train}}$  then flag episode as training-eligible
7:   return STORE
8: end if
9: if  $u_{\text{stored}} \geq \tau_{\text{defer}}$  then
10:  retain for cycle-boundary re-evaluation; do not write
11:  return DEFERRED
12: end if
13: if  $p_{\text{ground}}^{\text{max}} < \tau_{\text{abs}}$  then
14:  return a principled refusal to the caller
15:  return ABSTAIN
16: end if
17: suppress output
18: return DISCARD

```

× outcome) distribution as a primary figure.

1.8 Retroactive Re-Verification

Each self-improvement cycle ends with a single sweep over the entire memory in which every stored episode is re-scored under the fine-tuned model and either promoted, pruned, or left untouched. This is Stage 8 in the pipeline and runs once per cycle boundary after fine-tuning (and is therefore distinct from the per-query retrieval-feedback update of Section 1.4, which writes to u_{stored} continuously as Tier 1 answers are consumed). The retroactive pass is the only place in the architecture where the memory’s stored quality estimates are reconciled with the model that now consumes them: without it, a stored composite computed by the Cycle-0 verifier would continue to drive routing for episodes that are now more or less trustworthy under the Cycle-1 model.

Motivation: stored composites are snapshots, not invariants. Every u_{stored} value written to memory is a snapshot of the verifier’s view of a (question, answer) pair at the moment the episode was stored. Three of the nine signals (u_{token} , u_{dropout} , u_{internal}) read the model’s internal state directly and therefore drift whenever the weights drift; two more (s_{avg} , h_{norm}) depend on stochastic decoding and drift with the model’s output distribution. The remaining grounding signals depend on retrieval against a fixed passage corpus and are stable, but the NLI entailment scores between those passages and the generated answer still shift if the answer text itself is regenerated under the new model. Taken together, the stored composite drifts under every fine-tune. Retroactive re-verification is the pass that makes the drift visible and acts on it.

Three-way branch. For each stored episode e with current composite $u_{\text{stored}}^{(t)}$, the pass computes a new composite $u_{\text{stored}}^{\text{new}}$ using the updated verifier and assigns the episode to one of three branches:

$$e \mapsto \begin{cases} \text{prune} & \text{if } u_{\text{stored}}^{\text{new}} < \tau_{\text{retro}}, \\ \text{promote} & \text{if } u_{\text{stored}}^{\text{new}} > u_{\text{stored}}^{(t)} \text{ and } u_{\text{stored}}^{\text{new}} \geq \tau_{\text{retro}}, \\ \text{keep} & \text{otherwise,} \end{cases} \quad \tau_{\text{retro}} = 0.50. \quad (1.35)$$

A pruned episode is hard-deleted from both the FAISS index and the metadata dictionary. A promoted episode has its entire nine-signal record (plus p_{contra} , the decision string, and the early-exit flag) overwritten with the new verifier output, and its composite raised to $u_{\text{stored}}^{\text{new}}$. A kept episode retains its stored composite unchanged and is flagged `retroverified = True` to record that the pass did see it this cycle.

Upward-only update rationale. The promotion rule is intentionally asymmetric: the composite can only be raised by re-verification, never lowered toward zero. The asymmetry is a design commitment, not a numerical accident. If re-verification were allowed to overwrite the stored composite in either direction, a single noisy verifier pass (stochastic decoding variance in s_{avg} , h_{norm} , and u_{dropout} is non-trivial even on unchanged weights) could demote a stable, high-quality episode below the Tier 1 retrieval band on a coin flip. The memory would then lose accumulated knowledge faster than it could acquire it. The upward-only rule treats the stored composite as a lower bound on the episode’s quality that retroactive passes may raise; the *only* downward movement allowed is the categorical prune at τ_{retro} , which is reserved for cases where the new composite is so low that the episode is beyond rescue. The threshold $\tau_{\text{retro}} = 0.50$ sits strictly above the original DEFER threshold $\tau_{\text{defer}} = 0.45$, so an episode that survives re-verification maintains a margin above what would merely have deferred it at first commit. This margin is deliberate: re-verification is allowed to be stricter than initial storage because the system had already chosen to keep the episode once and should not pay full storage cost for an episode that a re-scoring under current weights would not even have deferred.

Nine-signal refresh on promotion. When an episode is promoted, the update is not limited to the scalar composite. The full verifier record is overwritten: all nine signals, p_{contra} , the STORE/DEFER/ABSTAIN/DISCARD decision string, and the early-exit flag. Downstream consumers of the memory (the calibration diagnostics of Chapter 5, the atomic-fact-distribution analyses, any future ablation that needs to re-partition the memory by verifier decision) therefore always see the latest verifier view of a promoted episode, not a stale mixture of the storage-time record and a later-computed composite. Kept episodes, in contrast, do not refresh their signals; they carry their storage-time record unchanged, with only the `retroverified` flag updated. The asymmetry matches the composite’s: only promotion is a substantive update.

Retroverify versus retrieval feedback. The two memory-update mechanisms have disjoint responsibilities and should not be conflated. Retrieval feedback (Section 1.4, Equation 1.10) is a continuous, per-query EMA update that tracks how often an episode’s Tier 1 answer was accepted by the user or by downstream checks, and is therefore about *usage quality*. Retroactive re-verification is a discrete, per-cycle pass that recomputes

the composite from first principles under the current model weights, and is about *model-relative quality*. The two signals can disagree: an episode can be consistently accepted at Tier 1 (high retrieval-feedback u_{stored} movement) while simultaneously drifting below the retroverify threshold when the fine-tuned model now produces a different internal-state reading. In such a case retroverify wins, because the system’s correctness argument rests on what the *current* verifier says about the (question, answer) pair, not on historical usage alone. The code enforces this precedence by running retroverify after fine-tuning and allowing it to remove entries that retrieval feedback alone would have kept.

Abort handling. Retroactive re-verification is skipped on any cycle whose fine-tuning step was rolled back by the forgetting-ratio guard (Section 1.9). The reason is operational rather than conceptual: if the weights were restored to the previous cycle’s checkpoint, re-scoring with unchanged weights produces the same composites that were already stored, up to stochastic-decoding noise, and the pass would therefore pay full compute cost for a no-op update with a small probability of injecting noise through the promotion path. The orchestrator in `self_improvement.py` therefore guards the retroverify call with `if verify_fn is not None and not aborted`, and records the counters $n_{\text{retroverified}} = n_{\text{retropruned}} = 0$ on aborted cycles so that Chapter 5’s retroverify-dynamics figures can distinguish aborted cycles from cycles in which no episode moved.

Algorithm 1.5 Stage 8 retroactive re-verification (runs once per cycle boundary after successful fine-tuning).

Require: Memory store $\mathcal{M}$; updated verifier $V^{(t+1)}$; prune threshold $\tau_{\text{retro}} = 0.50$.

Ensure: Memory $\mathcal{M}$ updated in place; counters $n_{\text{up}}, n_{\text{rm}}$.

```

1:  $n_{\text{up}} \leftarrow 0; n_{\text{rm}} \leftarrow 0$ 
2:  $\text{snapshot} \leftarrow \text{list of entry IDs in } \mathcal{M}$  ▷ dict may shrink during iteration
3: for all  $\text{eid} \in \text{snapshot}$  do
4:    $e \leftarrow \mathcal{M}.\text{lookup}(\text{eid});$  if  $e = \perp$  continue
5:   try  $o \leftarrow V^{(t+1)}.\text{verify}(e.\text{question}, e.\text{answer});$  except continue
6:    $u^{\text{new}} \leftarrow o.u_{\text{stored}}$ 
7:   if  $u^{\text{new}} < \tau_{\text{retro}}$  then
8:      $\mathcal{M}.\text{remove}(\text{eid}); n_{\text{rm}} \leftarrow n_{\text{rm}} + 1$ 
9:   else if  $u^{\text{new}} > e.u_{\text{stored}}$  then
10:    overwrite  $e$ ’s nine signals,  $p_{\text{contra}}$ , decision, early-exit flag from  $o$ 
11:     $\mathcal{M}.\text{update}_u(\text{eid}, u^{\text{new}}); e.\text{retroverified} \leftarrow \text{TRUE}$ 
12:     $n_{\text{up}} \leftarrow n_{\text{up}} + 1$ 
13:   else
14:      $e.\text{retroverified} \leftarrow \text{TRUE}$  ▷ checked this cycle; no update
15:   end if
16: end for
17: return  $(n_{\text{up}}, n_{\text{rm}})$ 
```

The per-cycle pair $(n_{\text{up}}, n_{\text{rm}})$ is the load-bearing diagnostic of the retroactive pass and is reported directly in Chapter 5. The dynamics claim underlying CAEM rests on the expected pattern that n_{up} grows and n_{rm} stays small across cycles: stored answers that the cycle-0 verifier rated marginally are re-rated more confidently by a better-calibrated cycle-1 verifier and by a better-grounded cycle-1 model, which is exactly the compounding signal that distinguishes confidence-aware episodic memory from a static retrieval cache.

A trajectory in which n_{rm} grows instead, especially if concentrated on episodes stored in a specific earlier cycle, is the counter-signal: the memory is accumulating material that the current model no longer regards as verified, and the system is in a regime where retroactive pruning is doing more work than retroactive promotion.

1.9 Self-Improvement Loop

Stage 8 is the mechanism that closes the outer loop of Figure 1.1 and turns CAEM from a confidence-aware retrieval system into a self-improving one. At the end of each cycle it fine-tunes the base model on the reasoning chains of the high-quality episodes in the memory, regularised against the previous cycle’s weights so that new knowledge accumulates without erasing the capabilities the model already had. The stage is deliberately conservative: the training pool is smaller than the Tier 1 retrieval pool (by design, not by accident), the regulariser is a uniform L_2 anchor rather than a full Fisher-weighted penalty, and the post-training capability check is out-of-distribution. If the check fails, the cycle is aborted and the previous checkpoint is restored. These choices together are the safety envelope inside which the compounding dynamics of Section 1.8 are allowed to operate.

Training-pool curation. At the start of each cycle, the training pool is drawn from the memory by filtering on the composite:

$$\mathcal{D}_{\text{train}}^{(c)} = \{e \in \mathcal{M}^{(c)} \mid u_{\text{stored}}(e) \geq \tau_{\text{train}}\}, \quad \tau_{\text{train}} = 0.75. \quad (1.36)$$

The training threshold $\tau_{\text{train}} = 0.75$ is stricter than the STORE threshold $\tau_{\text{store}} = 0.65$ from Section 1.7. The 0.10 gap between them is an architectural commitment: a stored episode is allowed to serve as a Tier 1 retrieval candidate with a lower composite than it needs to serve as training data, because retrieval acts on the model’s output but training acts on the model’s weights and can cause persistent harm if the data is under-verified. The gap cleanly separates the Tier 1 quality bar from the training-pool quality bar. Within the eligible set, the training target is the episode’s `reasoning_chain` field rather than the `answer` field: fine-tuning on the full chain teaches the model *how* to reason toward the correct answer, not merely to memorise answer strings, and it is the mechanism by which Tier 3 retrieval-augmented reasoning is internalised across cycles. An implementation-level fallback (empty or very short chains are replaced by the answer string alone) prevents training on blank targets without changing the curation rule itself.

Episode-and-general training mix. The episode pool $\mathcal{D}_{\text{train}}^{(c)}$ is combined with a small fraction of general-domain QA pairs before fine-tuning. The mixing rule uses a fixed ratio $r_{\text{gen}} = 0.10$:

$$|\mathcal{D}_{\text{gen}}^{(c)}| = \min\left(\left\lceil \frac{|\mathcal{D}_{\text{train}}^{(c)}| \cdot r_{\text{gen}}}{1 - r_{\text{gen}}} \right\rceil, |\mathcal{D}_{\text{gen}}^{\text{avail}}|\right), \quad (1.37)$$

where $\mathcal{D}_{\text{gen}}^{\text{avail}}$ is the pool of general QA pairs available to the orchestrator. The ratio is computed relative to the episode count so that episodes remain at $(1 - r_{\text{gen}}) = 90\%$ of the final mix regardless of how large the episode pool has grown by the current cycle. The 10% general share is the minimal diversity buffer consistent with the continual-learning literature: it is small enough that the model’s per-cycle parameter movement

is dominated by the memory-derived signal (otherwise memory would not actually drive improvement) and large enough that the model is exposed to prompts outside the verified-episode distribution at every cycle (otherwise drift toward the memory’s topic distribution becomes a systematic risk).

L₂ anchor regularisation. The fine-tuning loss is a standard sequence-level cross-entropy term plus an L₂ anchor on the parameter distance from the previous cycle:

$$\mathcal{L}(\theta) = \mathcal{L}_{\text{CE}}(f_{\theta}(q), y) + \frac{\lambda}{2} \|\theta - \theta_{\text{prev}}\|_2^2, \quad \lambda = 0.01. \quad (1.38)$$

Here θ_{prev} are the weights snapshotted at the start of the current cycle (equivalently, the accepted checkpoint of the previous cycle), and the anchor penalises every parameter uniformly for drifting away from that snapshot. This is the L₂ simplification of elastic weight consolidation doi:10.1073/pnas.1611835114: full EWC weights the per-parameter squared deviation by the diagonal of the Fisher information matrix, which requires an extra forward pass over the training set to estimate and roughly doubles the cost of a cycle. The uniform-weight simplification is a principled approximation when the Fisher matrix is close to uniform across parameters, a regime that holds reasonably well for a general-purpose seq2seq model fine-tuned on diverse QA chains. The thesis reports this choice as a documented design decision and the full-EWC comparison as a scheduled ablation in Chapter 5.

Out-of-distribution forgetting guard. After fine-tuning completes, the system evaluates the updated weights on a fixed 200-item MMLU split and compares the post-training accuracy against the pre-training baseline:

$$\rho_{\text{ret}}^{(c)} = \frac{\text{MMLU}(\theta^{(c)})}{\text{MMLU}(\theta^{(c-1)})}, \quad \text{abort iff } \rho_{\text{ret}}^{(c)} < \rho_{\text{min}}, \quad \rho_{\text{min}} = 0.93. \quad (1.39)$$

The guard is relative (a ratio), not absolute, so a model that enters the cycle with a weak MMLU baseline does not trigger an unwarranted abort merely because the baseline was already low. The benchmark is deliberately out-of-distribution: MMLU’s four-choice academic-subject questions are never in the training pool and share no topical overlap with the memory’s accumulated QA distribution. This choice follows directly from the continual-learning framing of doi:10.1073/pnas.1611835114: catastrophic forgetting in the continual-learning sense manifests as loss of *unrelated* general capability, not as erosion of the target task, so an in-distribution probe (TriviaQA, NaturalQuestions) would be systematically insensitive to the failure mode the guard is supposed to detect. The same MMLU pass also feeds the RET axis of the Composite Evaluation Score (Chapter 3), so the guard incurs no additional evaluation compute: one benchmark pass serves simultaneously as the rollback trigger and as the reported general-capability retention metric.

Rollback and abort semantics. If $\rho_{\text{ret}}^{(c)} < \rho_{\text{min}}$, the weights are restored to θ_{prev} , the cycle is flagged aborted in the cycle record, and the retroactive re-verification pass of Section 1.8 is skipped. The memory is not rolled back: episodes collected during the cycle remain stored, because their Stage 5 verification was performed under the previous-cycle weights and is therefore unaffected by the abort. An aborted cycle is not a failure of the overall pipeline but a commitment to the more conservative of the two available

checkpoints; the next cycle restarts from θ_{prev} and sees an enlarged memory that includes the episodes just stored. This asymmetry (memory accumulates across aborts; weights do not) is deliberate: the abort path ensures that weight-level drift cannot compound across cycles, while memory-level knowledge persists and is available to future cycles even if the immediately-following fine-tune failed.

Checkpointing and multi-cycle trajectories. Each cycle saves a complete, independently-loadable checkpoint to a dedicated directory, including the model weights (or the restored θ_{prev} if the cycle aborted), the cycle metadata, and the random seed. The isolation has two operational consequences: a failed cycle cannot overwrite a successful predecessor, and any cycle’s state can be reloaded in isolation for post-hoc comparison, ablation replay, or reproducibility audits. The outer loop is configured for $C = 10$ cycles in the code, of which three to five are evaluated end-to-end in Chapter 5 given compute budget; the choice of evaluated cycle count is reported in Chapter 5 and does not change the loop mechanism. Across the evaluated trajectory, four quantities jointly characterise the cycle and are reported per-cycle in Chapter 5: the size of the training pool $|\mathcal{D}_{\text{train}}^{(c)}|$ (grows as the memory accumulates), the final training loss, the MMLU retention ratio $\rho_{\text{ret}}^{(c)}$ (tracked against the abort threshold), and the retroverify counters $(n_{\text{up}}^{(c)}, n_{\text{rm}}^{(c)})$ from Section 1.8. A healthy trajectory shows the training pool growing monotonically, the loss decreasing or stable, the retention ratio safely above ρ_{min} , and n_{up} outpacing n_{rm} .

The compounding claim that motivates the cycle structure is a prediction, not a construction: nothing in the algorithm forces the model to improve cycle-over-cycle. What the algorithm does force is that improvement, when it happens, is attributable to the two signals the architecture controls (the verified training pool curated from memory, and the retroactive re-verification that follows fine-tuning) rather than to either signal in isolation. The trajectory Chapter 5 reports therefore disaggregates the two: an ablation in which fine-tuning is disabled isolates the memory’s direct effect through Tier 1 retrieval, while an ablation in which retroverify is disabled isolates the fine-tuning’s direct effect on new generations. The joint configuration with both channels active is the CAEM-complete condition, and its cycle trajectory is the headline figure of the Chapter 5 results.

Algorithm 1.6 Stage 8 self-improvement cycle (one invocation per cycle boundary).

Require: Current model $\theta^{(c-1)}$; memory store $\mathcal{M}^{(c)}$; general pool $\mathcal{D}_{\text{gen}}^{\text{avail}}$; thresholds $\tau_{\text{train}} = 0.75$, $\rho_{\text{min}} = 0.93$; hyperparameters $\lambda = 0.01$, $r_{\text{gen}} = 0.10$, epochs $E = 3$, learning rate $\eta = 10^{-5}$, batch $B = 16$.

Ensure: Updated model $\theta^{(c)}$ (or restored $\theta^{(c-1)}$ on abort); cycle record.

```

1:  $\mathcal{D}_{\text{train}}^{(c)} \leftarrow \{e \in \mathcal{M}^{(c)} : u_{\text{stored}}(e) \geq \tau_{\text{train}}\}$  with target =  $e.\text{reasoning\_chain}$ 
2: if  $|\mathcal{D}_{\text{train}}^{(c)}| = 0$  then
3:   return skip cycle (no eligible episodes)
4: end if
5: sample  $\mathcal{D}_{\text{gen}}^{(c)}$  from  $\mathcal{D}_{\text{gen}}^{\text{avail}}$  of size given by Eq. 1.37
6:  $\mathcal{D}^{(c)} \leftarrow \text{shuffle}(\mathcal{D}_{\text{train}}^{(c)} \cup \mathcal{D}_{\text{gen}}^{(c)})$ 
7:  $\theta_{\text{prev}} \leftarrow \theta^{(c-1)}$  ▷ snapshot for L2 anchor and rollback
8:  $m_{\text{pre}} \leftarrow \text{MMLU}(\theta_{\text{prev}}; n = 200)$ 
9: for  $k = 1$  to  $E$  do
10:   for all minibatch  $B \subset \mathcal{D}^{(c)}$  do
11:     compute  $\mathcal{L}(\theta)$  by Eq. 1.38
12:     AdamW step with gradient-norm clip at 1.0
13:   end for
14: end for
15:  $m_{\text{post}} \leftarrow \text{MMLU}(\theta; n = 200)$ ;  $\rho_{\text{ret}} \leftarrow m_{\text{post}}/m_{\text{pre}}$ 
16: if  $\rho_{\text{ret}} < \rho_{\text{min}}$  then
17:    $\theta \leftarrow \theta_{\text{prev}}$  ▷ rollback; cycle aborted
18:   save checkpoint; mark aborted  $\leftarrow \text{TRUE}$ ; return
19: end if
20: save checkpoint; retroverify  $\mathcal{M}^{(c)}$  under new  $\theta$  (Algorithm 1.5)
21: return updated  $\theta^{(c)}$ 

```
